

URBANBLADE

Manual Técnico Completo

UrbanBlade — Sistema de Gestión de Barbería (Laravel 12 + MongoDB + Docker)

Arquitectura, modelos de datos, servicios, controladores, rutas, seguridad y reglas de negocio

Equipo UrbanBlade

07 de agosto de 2026

Contenido

Contenido.....	2
0. Introducción y alcance del documento.....	5
0.1 Roles del sistema y qué puede hacer cada uno	5
0.2 Módulos funcionales del sistema	5
1. Stack tecnológico completo	7
1.1 Backend.....	7
1.2 Frontend.....	7
1.3 Variables de entorno (.env.example) por categoría	8
2. Arquitectura y despliegue con Docker	10
2.1 Vista general de la arquitectura.....	10
2.2 Servicios de docker-compose.yml.....	10
2.3 Dockerfile — construcción de la imagen	10
2.4 Automatización operativa.....	11
3. Modelos de datos (26 modelos Eloquent sobre MongoDB).....	13
3.1 El reto de los roles sobre MongoDB	13
4. Capa de servicios de dominio (app/Services/).....	22
5. Controladores (app/Http/Controllers/)	26
5.1 Controladores web por módulo.....	26
5.2 Controladores de API (prefijo v1)	27
6. Sistema de notificaciones (app/Notifications/)	29
7. Estructura de rutas.....	31
7.1 routes/web.php	31
7.2 routes/api.php (todo bajo el prefijo v1)	31
8. Vistas y frontend (Blade + Alpine.js)	33
8.1 Sistema de navegación.....	33
8.2 Componentes reutilizables (resources/views/components/)	33
8.3 Patrón visual y accesibilidad	34
9. Seeders y estructura de la base de datos	36
9.1 Patrones específicos de MongoDB en los seeders	37
10. Máquina de estados y reglas de negocio centrales	39
10.1 Ciclo de vida de una cita (AppointmentStatusService)	39
10.2 Notificación en cascada (AppointmentNotifier)	39
10.3 Niveles de lealtad (LoyaltyService)	39

10.4 Flujo de pedidos (tienda vs. add-on de cita).....	40
10.5 Despacho de campañas de marketing	40
10.6 Tarjeta de membresía digital	40
10.7 No-show automático y recordatorios	40
11. Seguridad	42
11.1 Autenticación y autorización	42
11.2 Protección de rutas y CSRF	42
11.3 Datos sensibles.....	42
11.4 Rate limiting	42
11.5 Monitoreo	43
12. Manejo de errores, logging y rendimiento	44
12.1 Excepciones de dominio	44
12.2 Notificaciones resilientes	44
12.3 Colas y procesamiento asíncrono	44
12.4 Concurrencia en inventario.....	44
12.5 Caché y colas sobre Redis	44
12.6 Estado actual de pruebas automatizadas	44
13. Configuración, comandos artisan y tareas programadas	46
13.1 Archivos de configuración relevantes (config/)	46
13.2 Comandos artisan personalizados (app/Console/Commands/)	46
13.3 Tareas programadas (routes/console.php)	46
14. Guía de resolución de problemas comunes.....	48
15. Glosario de términos técnicos del proyecto	50
16. Anexos.....	51
Anexo A — Checklist de despliegue a producción.....	51
Anexo B — Resumen de puertos expuestos	51
Anexo C — Resumen de roles y permisos.....	51
17. Ejemplos de peticiones a la API (v1)	53
17.1 Autenticación	53
17.2 Consultar disponibilidad	53
17.3 Crear una cita (autenticado)	53
17.4 Cambiar el estado de una cita.....	53
17.5 Registrar un pago	53
17.6 Feed social.....	53
18. Convenciones de código y guía de extensión	55

18.1 Convenciones generales	55
18.2 Cómo agregar una nueva notificación	55
18.3 Cómo agregar un nuevo tipo de reporte	55
18.4 Cómo agregar un nuevo estado o transición de cita	55

0. Introducción y alcance del documento

UrbanBlade es un sistema integral de gestión para barbería: reservas y agenda de citas, cobro y facturación, programa de fidelización con niveles y puntos, tienda de productos, muro social/portafolio de barberos, notificaciones multicanal (correo, in-app, SMS/WhatsApp), campañas de marketing con tracking de apertura/clic, tarjeta de membresía digital con QR, y un chatbot con inteligencia artificial (proveedor intercambiable entre un modelo local vía Ollama y Gemini en la nube).

Este manual documenta la arquitectura completa de la aplicación web (Laravel 12 + PHP 8.4 + MongoDB, desplegada con Docker) a nivel de código: modelos de datos, capa de servicios, controladores, rutas, notificaciones, vistas, seeders, reglas de negocio, seguridad, manejo de errores, rendimiento y despliegue. Está pensado como referencia para cualquier desarrollador que se incorpore al proyecto, para auditoría técnica externa, y como base de mantenimiento a largo plazo.

0.1 Roles del sistema y qué puede hacer cada uno

Rol	Alcance funcional
Administrador	Control total: usuarios, barberos, servicios, productos/inventario, configuración del negocio, reportes, campañas de marketing, bitácora de auditoría, respaldo de base de datos, mantenimiento del sistema.
Recepcionista	Operación diaria: gestión de citas (incluyendo walk-in), cobro de citas y pedidos, gestión de clientes, consulta y ajuste de inventario, bandeja de entrega de pedidos.
Barbero	Autogestión: su propia agenda del día, aprobar/rechazar/completar sus citas, editar su perfil y horario semanal, publicar en su portafolio (muro social).
Cliente	Autoservicio: reservar y cancelar sus citas, ver su historial y facturas, tarjeta de membresía con puntos/nivel, comprar en la tienda y ver sus pedidos, explorar y reaccionar en el muro social de barberos.

0.2 Módulos funcionales del sistema

- Citas y agenda — reserva, aprobación, calendario (FullCalendar), walk-in, máquina de estados estricta.
- Cobro y facturación — pagos por cita, propinas, recibo/factura en PDF, integración Stripe.
- Fidelización — niveles (Nuevo/Regular/VIP/Leyenda), puntos, canje, sorteo mensual, tarjeta de membresía con QR.
- Tienda y pedidos — catálogo de productos, carrito, checkout, add-ons dentro de una cita, bandeja de entrega.
- Inventario — productos de uso interno vs. venta, movimientos de stock, alertas de reorden.
- Muro social — portafolio de trabajos por barbero, comentarios, reacciones, guardados.
- Notificaciones — correo con marca, centro in-app, SMS/WhatsApp vía Twilio, preferencias por usuario.
- Campañas de marketing — envío inmediato o programado, segmentación por nivel de lealtad, tracking de apertura/clic.

- Chatbot con IA — asistente conversacional con memoria de contexto, aprendizaje y proveedor de IA intercambiable.
- Reportes y analítica — ingresos, citas, inventario, clientes, predicciones (demanda, servicios populares, horas pico).
- Administración y seguridad — roles/permisos, bitácora de auditoría, respaldo de base de datos, modo mantenimiento.

Convención importante: toda la persistencia usa MongoDB, no una base de datos relacional. Varios patrones de este manual (roles embebidos en el usuario, relaciones resueltas por consulta manual, seeders con inserción masiva, servicios que evitan `hasRole()`) existen específicamente para sortear limitaciones de paquetes de Laravel diseñados originalmente para SQL (como Spatie Permission) cuando corren sobre Mongo.

 ESPACIO PARA CAPTURA DE PANTALLA

Pantalla de inicio (landing page) de UrbanBlade — vista pública.

 ESPACIO PARA CAPTURA DE PANTALLA

Pantalla de acceso (login) del sistema.

1. Stack tecnológico completo

1.1 Backend

Componente	Versión / detalle
Lenguaje	PHP 8.4 (imagen Docker php:8.4-fpm; composer.json declara ^8.2 como mínimo soportado)
Framework	Laravel 12.x
Base de datos	MongoDB — paquete mongodb/laravel-mongodb ^5.0 (toda la app usa Mongo, sin MySQL/PostgreSQL)
Roles y permisos	spatie/laravel-permission ^7.2, con overrides propios para MongoDB (ver sección 3.1 y 10)
Bitácora de auditoría	spatie/laravel-activitylog ^5.0 — colección activity_log, adaptado a Mongo
Pagos con tarjeta	stripe/stripe-php ^20.3 (PaymentIntents + webhook con verificación de firma)
Generación de PDF	barryvdh/laravel-dompdf ^3.1 (facturas, recibos, tarjeta de socio)
Códigos QR	endroid/qr-code ^6.1 (QR de la tarjeta de membresía, SVG y PNG data-URI)
Exportes	maatwebsite/excel ^3.1 (reportes descargables)
Monitoreo de errores	sentry/sentry-laravel ^4.25 (captura de excepciones en producción)
Documentación de API	knuckleswtf/scribe (autogenera documentación desde las rutas y anotaciones)
Autenticación web	laravel/breeze ^2.4 (scaffolding de login/registro/reset)
Herramientas dev	laravel/pint (formateo), laravel/sail, laravel/pail (logs en tiempo real), laravel/boost
Colas	Redis (queue driver), procesadas por el contenedor worker

1.2 Frontend

El proyecto NO usa un framework SPA (React/Vue) para la web principal — es Blade renderizado en servidor, con Alpine.js para interactividad ligera (dropdowns, modales, paleta de comandos, flip de tarjeta, toasts). Esta decisión reduce la complejidad de build y mantiene el tiempo de carga inicial bajo, a costa de menos interactividad tipo SPA — un balance razonable para un sistema administrativo interno más que un producto de consumo masivo.

Componente	Versión / uso
Build tool	Vite ^8.0.16 + laravel-vite-plugin ^3.1.0
CSS	TailwindCSS ^3.1.0 + @tailwindcss/forms + @tailwindcss/vite
Interactividad	Alpine.js ^3.4.2 + @alpinejs/intersect (animaciones al hacer scroll)

Componente	Versión / uso
Calendario	@fullcalendar/{daygrid,timegrid,interaction,vue3} — usado en la agenda de citas
Gráficas del dashboard	Chart.js — usado en los 4 dashboards por rol
HTTP cliente-servidor	axios (llamadas AJAX desde blade/alpine sin recargar página)
Calidad de código JS	eslint + prettier + husky (git hooks) + lint-staged

Nota sobre el cliente móvil: el package.json también incluye react, react-dom, react-native, react-native-web y expo — indicando un cliente móvil (Expo/React Native) que consume la API v1 documentada en la sección 7.2, fuera del árbol web principal documentado en detalle aquí.

1.3 Variables de entorno (.env.example) por categoría

Aplicación

- APP_NAME, APP_ENV, APP_KEY, APP_DEBUG, APP_URL — configuración base de Laravel; APP_DEBUG debe ser false en producción.
- APP_LOCALE=es — idioma de la aplicación.

Passwords de seeders (opcionales)

- ADMIN_SEED_PASSWORD, RECEPTION_SEED_PASSWORD, BARBER_SEED_PASSWORD, CLIENT_SEED_PASSWORD — si se omiten, cada seeder genera una password aleatoria segura y la imprime UNA sola vez en la consola al sembrar; esto evita hardcodear credenciales en el código fuente.

Base de datos

- DB_CONNECTION=mongodb, MONGODB_URI (formato Atlas mongodb+srv://usuario:password@cluster), MONGO_DATABASE=barber_db.

Sesión, caché y colas

- SESSION_DRIVER=database (las sesiones se guardan en una colección de Mongo, no en archivos ni cookies firmadas solamente), SESSION_LIFETIME=120 minutos.
- QUEUE_CONNECTION=redis, CACHE_STORE=redis, REDIS_CLIENT=phpredis.

Correo

- MAIL_MAILER=log por defecto (los correos se escriben en el log en vez de enviarse — útil en desarrollo).
- Para Gmail SMTP real: MAIL_SCHEME=smtp (puerto 587, STARTTLS) o smtps (puerto 465, TLS implícito) — Laravel 12 rechaza explícitamente el esquema tls a secas, un detalle no obvio que causó un incidente documentado en la Unidad VI del módulo de analítica.

Almacenamiento

- Variables de AWS S3 disponibles pero vacías por defecto — el almacenamiento de archivos usa el disco local a menos que se configure explícitamente.

Chatbot con IA

- `CHATBOT_AI_PROVIDER=ollama` (modelo local, gratis, corre en el host) o `gemini` (modelo en la nube, requiere API key).
- `OLLAMA_URL=http://host.docker.internal:11434`, `OLLAMA_MODEL=qwen2.5:3b`, `OLLAMA_TIMEOUT=90` (segundos), `OLLAMA_KEEP_ALIVE=30m` (mantiene el modelo cargado en memoria).
- `GEMINI_API_KEY` — solo necesaria si se cambia el proveedor a Gemini.



ESPACIO PARA CAPTURA DE PANTALLA

Archivo `.env.example` abierto en el editor, con las secciones comentadas visibles.

2. Arquitectura y despliegue con Docker

2.1 Vista general de la arquitectura

La aplicación sigue una arquitectura monolítica modular: un solo código base Laravel expone tanto la interfaz web (Blade + Alpine) como una API REST versionada (v1) para el cliente móvil. Todo el estado se persiste en una única base de datos MongoDB (Atlas en producción). Las tareas asíncronas (envío de correos, generación de PDFs, notificaciones) se procesan por colas Redis en un contenedor worker separado, y las tareas programadas (recordatorios, resúmenes diarios, limpieza de tokens) corren en un contenedor scheduler independiente — separar estos dos procesos del servidor web principal evita que un envío de correo lento bloquee una petición HTTP de un usuario.

2.2 Servicios de docker-compose.yml

Los servicios app, worker y scheduler comparten una imagen local (barber-app:latest) y una configuración común mediante una ancla YAML (x-app-env): mismo archivo .env, servidores DNS 8.8.8.8/8.8.4.4, y montan tanto el código fuente como un volumen nombrado vendor_data compartido (evita reinstalar las dependencias de Composer en cada contenedor por separado, ahorrando tiempo de arranque).

Servicio	Rol	Detalle
app	PHP-FPM	extra_hosts host.docker.internal (permite que el chatbot llegue a Ollama corriendo nativo en el host, aprovechando GPU si existe); healthcheck nc -z localhost 9000; depende de redis healthy
web	Nginx (nginx:alpine)	puerto 8000:80 expuesto al host; monta .docker/nginx/default.conf; depende de app healthy
worker	Cola de trabajos	php artisan queue:work --verbose --tries=3 --timeout=90; healthcheck propio vía pgrep -f 'artisan queue:work' (no usa el healthcheck de puerto 9000 heredado del Dockerfile, porque este contenedor no escucha ese puerto)
scheduler	Tareas programadas	php artisan schedule:work (loop continuo); healthcheck vía pgrep -f 'artisan schedule:work'
mailpit	Correo de pruebas	Servidor SMTP de desarrollo con UI web; puertos 8025 (interfaz) y 1025 (SMTP)
redis	Caché y colas (redis:alpine)	healthcheck redis-cli ping

Todos los servicios comparten una red bridge interna llamada internal, aislada del resto del host salvo los puertos explícitamente publicados (8000, 8025, 1025).

2.3 Dockerfile — construcción de la imagen

- Imagen base: php:8.4-fpm (Debian slim con PHP-FPM preinstalado).
- Instala dependencias de sistema necesarias para las extensiones PHP: git, libpng-dev, libjpeg-dev, libwebp-dev, libzip-dev, libicu-dev, entre otras.

- Instala extensiones PHP vía `docker-php-ext-install`: `pdo`, `zip`, `exif`, `pcntl`, `gd`, `intl`; y vía PECL: `mongodb` (driver nativo de MongoDB) y `redis`.
- Copia el binario de Composer desde la imagen oficial `composer:latest` (multi-stage build, evita instalar Composer completo en la imagen final).
- `composer install --no-scripts` — deliberadamente evita ejecutar los scripts de post-instalación de Composer (que normalmente correrían comandos `artisan`) porque durante el build de la imagen las variables de entorno de conexión a MongoDB todavía no están disponibles; el `entrypoint` se encarga de eso al arrancar el contenedor, cuando el `.env` ya está montado.
- Ajusta los permisos de `storage/` y `bootstrap/cache/` al usuario `www-data` (necesario para que Laravel pueda escribir logs, caché de vistas compiladas y archivos subidos).
- Expone el puerto 9000 (PHP-FPM) y define un `healthcheck` `nc -z localhost 9000`.
- `ENTRYPOINT /entrypoint.sh` (script en `.docker/entrypoint.sh`) — ejecuta los comandos post-arranque (migraciones, cachés, etc.) antes de arrancar PHP-FPM.

2.4 Automatización operativa

Makefile

Centraliza los comandos Docker más usados en el día a día: `build`, `up` (`-d`, en segundo plano), `down`, `restart`, `logs` (`-f`, en vivo), `ps` (estado de contenedores), `shell` (abre `bash` dentro de app como el usuario `laravel`), `migrate`, `seed`, `setup` (`build + up + migrate`, e imprime las URLs finales de la app y de Mailpit), `clean` (`down --remove-orphans --volumes`, para un reinicio completamente limpio).

setup.ps1

Script de PowerShell para levantar el proyecto en Windows con Docker Desktop, pensado para que un nuevo integrante del equipo pueda arrancar todo con un solo comando: verifica que Git, Docker y Node.js estén instalados; clona o actualiza el repositorio (acepta parámetros `RepoUrl/Branch/TargetDir`); verifica que exista un archivo `.env` real (deliberadamente NO genera uno falso ni sobrescribe uno existente, ya que el repositorio es público y el `.env` real con credenciales se comparte por un canal separado y seguro); levanta los contenedores con Docker Compose; compila los assets de frontend con Node.js directamente en el host (no dentro de Docker, para aprovechar cachés de npm del sistema); e imprime las URLs finales de acceso.

 ESPACIO PARA CAPTURA DE PANTALLA

Salida de "docker compose ps" mostrando los 6 servicios en estado healthy.

 **ESPACIO PARA CAPTURA DE PANTALLA**

Salida de "make setup" o del script setup.ps1 completando la instalación end-to-end.

3. Modelos de datos (26 modelos Eloquent sobre MongoDB)

Todos los modelos extienden `MongoDB\Laravel\Eloquent\Model`, salvo `User` que extiende `MongoDB\Laravel\Auth\User` (necesario para la autenticación de Laravel). Cada colección de MongoDB corresponde a un modelo. A continuación se documenta cada uno en detalle: campos principales, relaciones y métodos notables — junto con las particularidades de trabajar con estos modelos sobre una base de datos documental en vez de relacional.

3.1 El reto de los roles sobre MongoDB

`spatie/laravel-permission` fue diseñado originalmente para bases de datos relacionales: la asignación de roles a un usuario se modela como una relación `MorphToMany` a través de una tabla pivote (`model_has_roles`). MongoDB no tiene tablas pivote nativas, y aunque el paquete `mongodb/laravel-mongodb` intenta emular esta relación, en la práctica el método `hasRole()` de Spatie resulta poco confiable sobre Mongo: en ciertos escenarios de carga (`loadMissing/eager loading`) puede devolver una colección vacía incluso cuando el usuario sí tiene el rol asignado.

Por esa razón, el modelo `User` mantiene además un campo `role_id` embebido directamente como array de strings en el propio documento del usuario (poblado en paralelo al asignar el rol), y expone dos métodos propios y confiables: `roleNames()` (consulta directa a la colección roles filtrando por los IDs en `role_id`) y `hasRoleName($nombre)` (envuelve `roleNames()->contains($nombre)`). El middleware de autorización de la aplicación (`role.custom / permission.custom`, ver sección 5) usa estos métodos en vez de `hasRole()/hasAnyRole()` de Spatie para garantizar consistencia.

User

Colección principal de autenticación.

Campo	Detalle
<code>name</code>	string
<code>email</code>	string, único
<code>password</code>	string, hasheado (cast automático)
<code>expo_push_token</code>	string, nullable — token de notificaciones push móviles
<code>notification_preferences</code>	array — preferencias por canal y tipo de notificación
<code>role_id</code>	array de strings — IDs de rol embebidos (ver 3.1)

Traits: `HasRoles` (Spatie), `LogsActivity`, `Notifiable`, `SoftDeletes`. Relaciones: `notifications` (`morphMany DatabaseNotification`), `barberProfile/clientProfile` (`hasOne Barber/Client`), `createdPayments` (`hasMany Payment` vía `created_by`), `inventoryMovements`, `mobileApiTokens`, `comments`, `reactions`, `savedWorks` (`hasMany`). Métodos clave: `issueMobileApiToken()` (genera un token de API móvil y guarda su hash SHA-256, nunca el token en claro), `notificationPreferences()` (fusiona valores por defecto con preferencias legadas y propias), `wantsNotificationChannel($canal)`, `routeNotificationForTwilio()`, `roleNames()/hasRoleName()`.

Appointment

El corazón operativo del sistema — una cita agendada.

Campo	Detalle
client_id, barber_id, service_id	referencias a otras colecciones
fecha	fecha de la cita
hora_inicio, hora_fin	horario de la cita
estado	pendiente confirmada en_proceso completada cancelada no_asistio
notas	texto libre
metodo_pago	string, nullable hasta que se cobra
precio_cobrado	decimal — puede diferir del precio de catálogo por descuentos
productos	array — add-ons de producto agregados en la reserva
motivo_reagendamiento, cancelada_en	auditoría de cambios
code	código público único (trait HasPublicCode)
confirmation_sent_at, reminder_24h_sent_at, reminder_2h_sent_at, cancellation_notified_at	marcas de tiempo para evitar reenviar la misma notificación dos veces

Traits: HasPublicCode, LogsActivity, SoftDeletes (las citas nunca se borran físicamente, solo se marcan).
Relaciones: client/barber/service (belongsTo), payments/inventoryMovements (hasMany).

Barber

Perfil profesional de un usuario con rol barbero.

Campo	Detalle
user_id	referencia a User
nombre	nombre público
especialidad	especialidad principal (singular)
especialidades	STRING separado por comas — contrato explícito, NO un array (romper esto rompe vistas y controladores en todo el proyecto)
telefono, foto, descripcion	datos de perfil
activo	bool
slug	para URLs públicas amigables

Trait HasSlug. Relaciones: user, appointments, works (hasMany, fk barbero_id — nótese que apunta a users._id, no a barbers._id, una asimetría intencional del esquema), schedules. comments() se resuelve con una consulta manual por los IDs de sus works, no una relación Eloquent directa.

BarberReview

Reseña de un cliente sobre un barbero.

Campo	Detalle
barber_id, client_id	referencias
rating	entero
comment	texto

belongsTo barber/client. Se crea desde CatalogController::storeReview (API) o ClientBarberController::storeReview (web).

BarberSchedule

Horario semanal de disponibilidad de un barbero.

Campo	Detalle
barber_id	referencia
day_of_week	convención Laravel 0=domingo...6=sábado
start_time, end_time	horario del turno
is_working	bool — permite marcar un día como descanso sin borrar el registro

belongsTo barber. Es la fuente de verdad que usa AppointmentService::getAvailableSlots() para calcular horarios disponibles.

BarbershopSetting

Configuración global del negocio — documento único (singleton lógico).

Campo	Detalle
nombre, logo, direccion, telefono	datos de identidad
horario_apertura, horario_cierre	horario oficial, usado como fallback si un barbero no tiene horario propio
politica_cancelacion	horas de anticipación requeridas
redes_sociales	array
datos_bancarios	array
maintenance_mode	bool — activa la pantalla de mantenimiento para todos salvo administradores

Se accede siempre con first() (o se crea uno con valores por defecto si no existe, ver CheckMaintenanceMode).

Campaign

Una campaña de marketing (promoción).

Campo	Detalle
titulo, cuerpo	contenido del mensaje
cta_label, cta_url	botón de llamada a la acción
segmento	'todos' o un nivel de lealtad específico
destinatarios	entero — se llena al despachar

Campo	Detalle
enviado_por	referencia al admin que la creó
estado	'programada' 'enviada'
programada_para, enviada_en	fechas
opened_by, clicked_by	arrays de IDs de usuario — tracking de engagement

Métodos: opensCount(), clicksCount(), rate() (tasa de apertura/clic sobre destinatarios).

Client

Perfil de cliente de un usuario con rol cliente.

Campo	Detalle
user_id	referencia
telefono, fecha_nacimiento	datos personales
preferencias_notificacion	array, campo legado — superado por User.notification_preferences
slug	URL amigable
nivel	'nuevo' 'regular' 'vip' 'leyenda'
puntos	entero, saldo de puntos de lealtad
total_citas	contador de citas completadas

Trait HasSlug. Los accessors de nivel/puntos/total_citas tienen valores por defecto seguros (nivel='nuevo', puntos=0, total_citas=0) para clientes recién creados. Relaciones: user, appointments, loyaltyTransactions.

Comment

Comentario en una publicación del muro social.

Campo	Detalle
work_id, user_id	referencias
comment	texto
rating	opcional

belongsTo user/work.

DatabaseNotification

Respaldo del canal 'database' de las notificaciones de Laravel.

Campo	Detalle
id	string, PK no incremental (UUID)
type	clase de la notificación
notifiable_id/type	morphTo — a quién pertenece

Campo	Detalle
data	array — contenido serializado de la notificación
read_at	datetime, nullable

Métodos `markAsRead()/markAsUnread()`, scopes `read()/unread()`. Alimenta el centro de notificaciones in-app (ícono de campana).

InventoryMovement

Registro de entrada o salida de stock de un producto.

Campo	Detalle
product_id	referencia
tipo	'entrada' 'salida'
cantidad	entero
motivo	texto — por qué se movió el stock
appointment_id	opcional — si el movimiento vino de un add-on de cita
user_id	quién lo registró
fecha	fecha del movimiento

Trait `LogsActivity`. belongsTo `product/appointment/user`. Es la fuente auditable detrás de `InventoryService::registerMovement()`.

LoyaltyTransaction

Movimiento de puntos de fidelización.

Campo	Detalle
client_id	referencia
tipo	'ganado' 'canjeado'
puntos	entero (positivo o negativo según tipo)
descripcion	texto
referencia_id	opcional — enlaza a la cita/canje que originó el movimiento

belongsTo `client`. Escrito por `LoyaltyService::awardCitaPoints()/awardResenaPoints()/redeemPoints()`.

MobileApiToken

Token de autenticación de la app móvil (no usa Sanctum).

Campo	Detalle
user_id	referencia
name	etiqueta del dispositivo/sesión
token_hash	SHA-256 del token — el token en claro nunca se persiste

Campo	Detalle
abilities	array — alcance de permisos del token
last_used_at, expires_at	control de expiración y auditoría de uso

belongsTo user. Validado por el middleware mobile.auth en cada petición de la API v1.

Order

Un pedido de productos (tienda o add-on de cita).

Campo	Detalle
client_id	referencia
folio	código único legible, formato P-XXXXXX
items	array embebido: product_id, nombre, precio, cantidad, subtotal por línea
total	decimal(2)
estado	'pendiente' 'entregado' 'cancelado'
tipo	'tienda' 'cita'
appointment_id	solo si tipo='cita'
metodo_pago, entregado_en, notas	datos complementarios

Método itemCount(). Creado y gestionado por OrderService (ver sección 10.4).

Payment

Un cobro registrado sobre una cita.

Campo	Detalle
appointment_id	referencia
monto	decimal(2)
metodo_pago	efectivo tarjeta transferencia, etc.
propina	decimal(2)
comprobante_pdf	ruta del PDF generado
created_by	quién procesó el cobro

Trait LogsActivity. belongsTo appointment, creator (User vía created_by). Dispara PaymentReceiptNotification al crearse.

Permission (Spatie, adaptado)

Permiso granular del sistema de autorización.

Campo	Detalle
name, guard_name	identificador del permiso

El constructor fuerza guard_name para evitar inconsistencias. Métodos estáticos findById/findByName/findOrCreate. Relaciones roles()/users() adaptadas a Mongo.

Product

Artículo de inventario (uso interno o venta al cliente).

Campo	Detalle
nombre, categoria, descripcion	datos del catálogo
precio_compra, precio_venta	decimal(2) — el margen se calcula como venta-compra
stock_actual, stock_minimo	control de inventario
tipo	'uso_interno' 'venta'
imagen	ruta de la foto del producto

SoftDeletes + LogsActivity. hasMany movements (InventoryMovement).

RaffleResult

Resultado del sorteo mensual de lealtad.

Campo	Detalle
client_id	referencia
mes	formato 'YYYY-MM'
premio	descripción del premio
nivel_ganador	nivel de lealtad del ganador

belongsTo client. Generado por el comando loyalty:draw-raffle.

Reaction

"Me gusta" de un usuario sobre una publicación del muro.

Campo	Detalle
work_id, user_id	referencias
type	tipo de reacción

belongsTo user/work.

Role (Spatie, adaptado)

Rol del sistema (administrador/recepcionista/barbero/cliente).

Campo	Detalle
name, guard_name	identificador del rol

Análogo a Permission: findById/findByName/findOrCreate, permissions()/users() adaptadas a Mongo.

SavedWork

Publicación guardada/marcada como favorita por un usuario.

Campo	Detalle
user_id, work_id	referencias

belongsTo user/work.

Service

Servicio de catálogo (corte, barba, combo, tratamiento, etc.).

Campo	Detalle
nombre, categoria	clasificación
precio	float
duracion_min	duración estimada
imagen, descripcion	datos de presentación
activo	bool — permite ocultar sin borrar
slug	URL amigable

Trait HasSlug. belongsToMany combos (pivot combo_service), hasMany appointments.

ServiceCombo

Paquete de varios servicios con descuento.

Campo	Detalle
nombre	nombre del combo
precio_combo	precio fijo del paquete
descuento	decimal(2), % respecto a la suma individual

belongsToMany services (relación inversa de Service.combos).

Work

Publicación del portafolio social de un barbero.

Campo	Detalle
barbero_id	referencia DIRECTA a users._id, no a barbers._id — asimetría intencional del esquema
title, description	contenido de la publicación
work_date	fecha del trabajo mostrado

belongsToMany barberUser (User), hasMany images/comments/reactions/saves. Métodos isReactedBy(\$user)/isSavedBy(\$user).

WorkImage

Foto o video adjunto a una publicación del muro.

Campo	Detalle
image	ruta del archivo
type	'image' 'video'
mime_type	tipo MIME real del archivo

Método isVideo(). belongsTo work.

Activity (Spatie Activitylog)

Registro de auditoría genérico.

Campo	Detalle
log_name, description	categoría y descripción del evento
subject_id/type	morphTo — sobre qué modelo ocurrió
causer_id/type	morphTo — quién lo causó
properties	array — datos adicionales del evento
batch_uuid	agrupa varios cambios de una misma operación

Colección activity_log. Scopes: inLog(), forSubject(), forCauser(), forEvent(), forBatch(). Usado por el trait LogsActivity en Appointment, InventoryMovement, Payment, Product y User — alimenta la pantalla de Bitácora (Log\ActivityLogController).

 ESPACIO PARA CAPTURA DE PANTALLA

Diagrama entidad-relación (ER) de las colecciones principales, o vista de MongoDB Compass mostrando las colecciones y sus conteos.

4. Capa de servicios de dominio (app/Services/)

La lógica de negocio vive en servicios dedicados, no en los controladores — estos quedan delgados y solo orquestan HTTP + autorización + llamada al servicio correspondiente. Esta separación permite reutilizar la misma lógica desde la web, la API móvil y los comandos artisan sin duplicar código, y facilita las pruebas unitarias al aislar la lógica de negocio del ciclo de vida HTTP.

Appointment/

- `AppointmentService` — calcula slots disponibles con `getAvailableSlots()`: usa el horario específico del barbero (`BarberSchedule`) si existe, o cae al horario global de la barbería (`BarbershopSetting`) como respaldo; bloquea horarios que ya pasaron con un margen de seguridad de 10 minutos; detecta y excluye horarios que se solaparían con citas ya existentes del mismo barbero.
- `createAppointment()/updateAppointment()` validan con `ensureNoOverlap()`: un cliente no puede tener más de una cita el mismo día (lanza `ClientAlreadyBookedException`) y un barbero no puede tener dos citas que se solapen en horario (lanza `AppointmentConflictException`) — ambas excepciones se traducen a mensajes de error claros en la UI.
- `AppointmentStatusService` — máquina de estados centralizada, única fuente de verdad de qué transiciones son legales (detalle completo en la sección 10.1).
- `AppointmentNotifier` — reparte notificaciones a cliente/barbero/personal en creación, cancelación y cambios de estado; estructurado para NUNCA romper el flujo principal si el envío de una notificación falla (captura la excepción y solo registra un warning en el log).

Business/

- `BusinessEventService` — envoltura ligera sobre el helper `activity()` de Spatie Activitylog, para registrar eventos de negocio con un conjunto de propiedades adicionales y el sujeto (modelo) asociado, de forma consistente en todo el proyecto.

Campaign/

- `CampaignDispatcher::audienceUserIds()` resuelve el segmento de una campaña ('todos' o un nivel de lealtad específico como 'vip') a la lista real de IDs de usuario a los que aplica.
- `segmentCounts()` calcula cuántos clientes hay en cada segmento posible, usado en la UI de creación de campaña para mostrar el alcance estimado antes de enviar.
- `dispatch()` construye la `PromotionNotification` y la envía en lotes (chunks) de 200 destinatarios a la vez, para no saturar la cola de Redis con miles de trabajos simultáneos; al terminar, marca la campaña como 'enviada' con el conteo real de destinatarios y la fecha/hora.

Cart/

- `CartService` — implementa el carrito de compras usando la sesión HTTP de Laravel (clave 'cart', no requiere autenticación previa para empezar a agregar productos). Al agregar una línea, respeta el `stock_actual` disponible del producto (no permite agregar más unidades de las que hay en existencia). Expone `updateQuantity()`, `removeItem()`, `count()` (total de artículos) y `total()` (suma monetaria).

Chatbot/

- `ChatbotContextService`, `ChatbotUserProfileService` — arman el contexto de la conversación (historial reciente, perfil del usuario que pregunta).

- `ChatbotExternalDataService` — trae datos reales del negocio (disponibilidad, precios, promociones) para que las respuestas del bot sean precisas y no inventadas.
- `ChatbotIntelligenceService` — orquesta la cascada de resolución: primero intenta responder con reglas/memoria, y solo si no puede, delega al proveedor de IA configurado.
- `ChatbotLearningService` — registra qué preguntas no se pudieron responder bien, para mejorar las reglas con el tiempo (aprendizaje supervisado manual, no un modelo que se reentrena solo).
- `GeminiService` y `OllamaService` — ambos implementan el contrato `Contracts\ChatbotAiProvider` (`isEnabled()`, `label()`, `generateResponseWithPrompt()`), lo que permite cambiar de proveedor de IA con solo una variable de entorno (`CHATBOT_AI_PROVIDER`), sin tocar el resto del código.
- `Concerns/BuildsBarberSystemPrompt` — trait compartido que construye el "system prompt" (instrucciones de personalidad y contexto) que se le da al modelo de IA antes de cada consulta.

Dashboard/

- `DashboardService` concentra los 4 dashboards por rol en una sola clase, con un método dedicado por rol: `adminMetrics()` (KPIs globales del negocio), `barberMetrics($barberId)` (agenda y desempeño personal), `receptionistMetrics()` (operación del día), `clientMetrics($clientId)` (historial y tarjeta de lealtad). Mantenerlos juntos facilita ver de un vistazo qué datos expone cada rol y evitar fugas de información entre roles.

Inventory/

- `InventoryService` — CRUD de productos a través de un repositorio (`ProductRepositoryInterface`, patrón repositorio para desacoplar el servicio del ORM concreto).
- `lowStockCount()` — cuenta cuántos productos están en o por debajo de su stock mínimo, usado en las alertas del dashboard admin.
- `registerMovement()` — el método más delicado del servicio: envuelve la operación en una transacción de base de datos con `lockForUpdate()` sobre el documento del producto, para evitar condiciones de carrera si dos operaciones intentan modificar el mismo stock simultáneamente (por ejemplo, dos ventas del mismo producto casi al mismo tiempo). Valida que haya stock suficiente antes de una salida (lanza `InsufficientStockException` si no).

Loyalty/

- `LoyaltyService` — define las constantes de negocio del programa de lealtad: `LEVELS` (umbrales de citas por nivel), `DISCOUNTS` (% de descuento por nivel), `LEVEL_LABELS` (nombres de marketing: Caballero/Regular/V.I.P/Leyenda). Ver tabla completa en la sección 10.3.
- `nivelFromCitas($totalCitas)` determina el nivel correspondiente a un conteo de citas; `nextLevel($nivelActual)` devuelve el siguiente nivel a alcanzar (para mostrar "te faltan N citas para VIP" en la UI).
- `awardCitaPoints()` otorga 10 puntos al completar una cita, recalcula el nivel, y si el cliente sube de nivel dispara `LoyaltyNotification`.
- `awardResenaPoints()` otorga 5 puntos al dejar una reseña.
- `redeemPoints($cantidad)` valida que el cliente tenga saldo suficiente ANTES de descontar puntos — nunca permite un saldo negativo.

Member/

- `MemberCardService` genera los datos de la tarjeta de membresía digital: `memberNumber()` deriva un número de socio legible y estable a partir del `ObjectId` único del usuario (formato "UB · XXXX

XXXX"), memberSince() toma el año de creación de la cuenta, y qrDataUri()/qrPngDataUri() generan un código QR (en formato SVG o PNG, codificado como data-URI listo para insertar en HTML o PDF) cuyo contenido es el payload UB|{id}|{name} — un formato simple que cualquier lector de QR de terceros podría, en teoría, decodificar para verificar identidad de socio.

Messaging/

- MessagingService — capa de envío de SMS/WhatsApp: sendSms() y sendWhatsapp() envuelven llamadas a la API de Twilio. Si las credenciales de Twilio no están configuradas en el entorno, el servicio SIMULA el envío y lo registra en el log en vez de fallar — esto permite que el resto del sistema de notificaciones funcione en desarrollo sin necesitar una cuenta de Twilio real.

Order/

- OrderService::place() — antes de descontar CUALQUIER stock, valida que TODOS los ítems del pedido tengan disponibilidad suficiente (evita el escenario de descontar 3 de 5 productos y luego fallar en el cuarto, dejando el inventario en un estado inconsistente). Registra un movimiento de inventario tipo 'salida' por cada línea, genera un folio único legible (P-XXXXXX), y crea el Order en estado inicial 'pendiente'.
- cancel() solo puede actuar sobre pedidos que sigan en estado 'pendiente' (un pedido ya entregado no se puede cancelar retroactivamente) y revierte el stock descontado con movimientos de tipo 'entrada'.

Payment/

- PaymentService — list()/create() de pagos, delegando la persistencia a un PaymentRepositoryInterface.
- StripePaymentService — createPaymentIntent(), retrievePaymentIntent(), confirmPayment() — envuelve el flujo de PaymentIntents de Stripe para cobros con tarjeta, cuya confirmación final llega de forma asíncrona vía el webhook (StripeWebhookController).

Prediction/

- PredictionService — capa de analítica ligera integrada directamente en la app (distinta de la sección Analítica avanzada, ver sección 2.4): predictIncome(), predictAppointments(), predictPopularServices(), predictPeakHours(), generateInsights() — cálculos estadísticos simples (promedios, tendencias) sobre los datos históricos de MongoDB, expuestos en el panel de administración para dar una primera capa de insights sin entrar a la pantalla de Analítica.

Report/

- ReportService — incomeReport(), appointmentsReport(), inventoryReport(), clientsReport() generan cada uno un reporte tabular filtrable por rango de fechas; reportByType(\$tipo) actúa como dispatcher para resolver dinámicamente cuál de los métodos anteriores invocar desde el controlador, evitando un switch/case repetido en cada punto de entrada (web y API).

Service/

- ServiceService — CRUD de servicios de catálogo vía repositorio, y categories() para listar las categorías distintas disponibles (usado en filtros de la UI).

 ESPACIO PARA CAPTURA DE PANTALLA

Estructura de carpetas de app/Services/ en el editor de código.

5. Controladores (app/Http/Controllers/)

Patrón de autorización de dos capas, aplicado consistentemente en todo el proyecto: middleware `role.custom:<roles>` (verifica el rol de Spatie vía `User::hasRoleName()`, no `hasRole()` — ver sección 3.1) y middleware `permission.custom:<permisos>` (verifica un permiso granular de Spatie de la misma forma confiable). Ambos middlewares tienen implementación propia dentro de `app/Http/Middleware/` precisamente para sortear la fragilidad de los métodos estándar de Spatie sobre MongoDB.

5.1 Controladores web por módulo

Módulo	Controlador(es) y acciones principales	Gate de autorización
Citas	Appointment\AppointmentController: index, searchClients (autocompletado para walk-in), walkIn (alta rápida), create/store/edit/update, calendar/calendarData (feed FullCalendar), updateStatus, destroy.	permission.custom:citas.gestionar
Autenticación	Auth/: AuthenticatedSessionController, RegisteredUserController, PasswordResetLinkController, NewPasswordController, ConfirmablePasswordController, PasswordController, EmailVerificationNotificationController, VerifyEmailCodeController — set de Breeze adaptado: la verificación de cuenta usa un código de 6 dígitos en vez del enlace firmado original (VerifyEmailController ya no se usa).	público / auth según acción
Barberos (gestión)	Barber\BarberController: index/performance/edit/show/update.	permission.custom:barberos.gestionar
Barbero (self-service)	Barber\BarberDashboardController: agenda, updateAppointmentStatus, editProfile/updateProfile, editSchedule/updateSchedule.	role.custom:barbero
Campañas	Campaign\CampaignController: index/send. Campaign\TrackingController: open/click (públicos, sin auth).	role.custom:administrador (excepto tracking)
Chatbot	Chatbot\ChatbotController: query (throttled), getHistory/clearHistory/getProfile/getLearningStats/trainFromHistory.	público (query) / auth (resto)
Cliente — carrito	Client\CartController: index/add/update/remove/checkout.	role.custom:cliente
Cliente — citas	Client\ClientAppointmentController: self-service de reserva/cancelación.	role.custom:cliente
Cliente — barberos	Client\ClientBarberController: index/show/storeReview.	role.custom:cliente
Cliente — gestión (admin)	Client\ClientController: CRUD administrativo de clientes.	permission.custom:clientes.gestionar
Cliente — facturas	Client\ClientInvoiceController: index/download (PDF).	role.custom:cliente

Módulo	Controlador(es) y acciones principales	Gate de autorización
Cliente — membresía	Client\MembershipController: card (tarjeta + QR + descarga PDF).	role.custom:cliente
Cliente — pedidos	Client\OrderController: index/cancel.	role.custom:cliente
Cliente — tienda	Client\StoreController: index (catálogo).	role.custom:cliente
Dashboard	Dashboard\DashboardController: index (dispatcher por rol). Dashboard\DatabaseBackupController: download.	auth / role.custom:administrador (back)
Inventario	Inventory\InventoryMovementController, ProductController: CRUD completo.	permission.custom:inventario.ver / inventario.gestionar
Bitácora	Log\ActivityLogController: index.	permission.custom:logs.ver
Notificaciones	Notification\NotificationController: index/markAllRead/markOneRead/preferences/updatePreferences/poll.	auth (todos los roles)
Pagos	Payment\PaymentController: index/create/store/destroy/downloadReceipt.	permission.custom:pagos.gestionar
Perfil	Profile\ProfileController: edit/update/destroy.	auth (todos los roles)
Bandeja de pedidos	Reception\OrderController: index/deliver/cancel/receipt.	permission.custom:pagos.gestionar / citas.gestionar
Reportes	Report\ReportController: index/export.	permission.custom:reportes.ver
Servicios	Service\ServiceController: index/publicIndex/create/store/edit/update/destroy.	permission.custom:servicios.gestionar (gestión); público (publicIndex)
Configuración	Setting\BarbershopSettingController: edit/update/toggleMaintenance.	permission.custom:configuracion.gesti
Social	Social\BarberPortfolioController: index/create/store/destroy. SocialController: feed/react/save/comment.	role.custom:barbero (portafolio) / auth+verified (interacciones)
Usuarios	User\UserController: CRUD completo.	permission.custom:usuarios.gestionar

5.2 Controladores de API (prefijo v1)

Api/ contiene los controladores de la API móvil, autenticada con el middleware propio mobile.auth (token Bearer emitido por User::issueMobileApiToken(), no Laravel Sanctum): AuthController, AppointmentController, AvailabilityController, BarberManagementController, BarberPortfolioController, BarberScheduleController, CatalogController (catálogo público de servicios/barberos + showBarber/storeReview), ChatbotManagementController, ClientController, DashboardController, InventoryController, LogController, NotificationController, PaymentController (incluye stripeIntent), PredictionController, ProfileController (incluye bio de barbero y push token), ReportController (export), ServiceManagementController, SettingController (incluye toggleMaintenance), SocialController, StripeWebhookController (sin CSRF, valida firma HMAC de Stripe), UserController.

Api/Admin/ (gate role.custom:administrador, todo bajo prefijo admin) contiene los equivalentes de panel administrativo sobre la API: BarberAdminController (getBarbers/show/getSchedule/getRegularClients/update/getPerformanceStats), ClientAdminController (getClients/show/getSegmentation/store/update/destroy/export), DashboardAdminController (getStats/getUpcomingAppointments/getRevenue/getAlerts/getMetrics), InventoryAdminController (getProducts/show/store/update/destroy/recordMovement/getMovements/getSummary/getLowStockProducts), ReportAdminController (generateRevenueReport/generateAppointmentsReport/generateInventoryReport/generateClientsReport/generateCustomReport/exportReport/listReports).

 **ESPACIO PARA CAPTURA DE PANTALLA**

Panel de administración mostrando el módulo de gestión de barberos.

 **ESPACIO PARA CAPTURA DE PANTALLA**

Documentación de API autogenerada por Scribe (resources/views/scribe/).

6. Sistema de notificaciones (app/Notifications/)

Todas las notificaciones implementan la interfaz ShouldQueue — se encolan en Redis en el momento de dispararse y son procesadas de forma asíncrona por el contenedor worker, para que el usuario que dispara el evento (por ejemplo, al confirmar una cita) no tenga que esperar a que el correo termine de enviarse. Todas respetan las preferencias de canal del usuario destino, consultadas vía `User::wantsNotificationChannel()`.

Notificación	Disparador	Canales	Contenido / detalle
AppointmentNotification	Creación, cancelación o cambio de estado de una cita (vía AppointmentNotifier)	database + mail + Twilio	Adjunta archivo .ics y enlace a Google Calendar en confirmaciones; incluye método toTwilio() propio para SMS/WhatsApp corto
DailySummaryNotification	Comando reports:daily-summary (cron diario 21:30)	database + mail (solo admin)	Array de estadísticas del día (etiqueta→valor) + enlace a reportes
InventoryLowStockNotification	Comando inventory:low-stock-alert (cron diario 09:00)	database + mail	Lista de productos bajo stock mínimo; marca "Agotado" si stock llega a 0
LoyaltyNotification	LoyaltyService::awardCitaPoints() al subir de nivel	database + mail	Nuevo nivel, nivel anterior, % de descuento ganado
OrderDeliveredNotification	Reception\OrderController::deliver()	database + mail	Detalle completo del pedido entregado (folio, items, total)
PaymentReceiptNotification	Creación de un Payment	database + mail	Factura PDF adjunta generada al vuelo (folio F-XXXXXX)
PromotionNotification	CampaignDispatcher::dispatch() o comando campaigns:promote	database + mail (requiere opt-in 'promociones')	Inyecta píxel de apertura y redirección de clic si viene de una Campaign, para medir engagement

Notificación	Disparador	Canales	Contenido / detalle
ReviewRequestNotification	AppointmentNotifier tras marcar 'completada' (delay de 2h)	database + mail	Enlace a la página del barbero para dejar reseña

Channels\TwilioChannel es un canal de entrega custom (no una notificación en sí): usa MessagingService para enviar SMS o WhatsApp, priorizando WhatsApp si el usuario lo activó explícitamente en sus preferencias. Requiere que la notificación implemente toTwilio() y que el modelo destino implemente routeNotificationForTwilio() (ya presente en User).

Todas las notificaciones de correo comparten la misma plantilla de marca (vista emails.message), parametrizada con accent (color de acento), badge (etiqueta visual del estado), title, rows (filas de datos clave-valor), ctaLabel/ctaUrl (botón de acción) y pixel (URL de tracking de apertura) — esto asegura consistencia visual entre las 8 notificaciones sin duplicar HTML/CSS de correo en cada una.

 ESPACIO PARA CAPTURA DE PANTALLA

Correo recibido de una notificación (por ejemplo, confirmación de cita con el diseño de marca).

 ESPACIO PARA CAPTURA DE PANTALLA

Centro de notificaciones dentro de la app (ícono de campana) y la pantalla de preferencias por canal.

7. Estructura de rutas

7.1 routes/web.php

- Públicas: / (landing), /mantenimiento (pantalla de mantenimiento), /equipo/{barber} (perfil público de barbero), /servicios (catálogo público), /t/o/{campaign}/{user} y /t/c/{campaign}/{user} (tracking de apertura/clic de correo de campaña, sin autenticación), /chatbot/query (con throttle para evitar abuso).
- La landing pública (/) está organizada en 7 secciones ancladas en una sola vista (Inicio, Servicios, Proceso, Maestros, Ubicación, Acceso, Reservar); los datos de servicios/barberos/estadísticas se calculan con `Cache::remember(..., 300, ...)` para no recalcularlos en cada visita anónima. La sección Ubicación embebe un mapa real (iframe de OpenStreetMap) en vez de un marcador estático, con enlace directo a Google Maps.
- `middleware(['auth'])`: historial de conversación, perfil, estadísticas de aprendizaje y limpieza de historial del chatbot.
- `/descubrir` (feed social, público) + `middleware(['auth','verified'])`: reaccionar/guardar/comentar en publicaciones; anidado dentro un grupo `role.custom:administrador` para alternar el modo mantenimiento del sitio.
- El bloque principal `middleware('auth')` anida sub-grupos según rol y permiso específico:
 - `role.custom:administrador,recepcionista` → `permission.custom:citas.gestionar` (walk-in, cambio de estado, calendario), `permission.custom:pagos.gestionar` (pagos + bandeja de entrega de pedidos de Reception\OrderController), `permission.custom:inventario.ver/inventario.gestionar`, `permission.custom:clientes.gestionar`.
 - `role.custom:administrador` (en solitario) → respaldos de base de datos, gestión de campañas, `permission.custom:reportes.ver`, `permission.custom:servicios.gestionar`, `permission.custom:inventario.gestionar` (productos), `permission.custom:usuarios.gestionar`, `permission.custom:barberos.gestionar` (incluye desempeño), `permission.custom:configuracion.gestionar`, `permission.custom:logs.ver`.
 - `role.custom:cliente` con prefijo de URL `/cliente` y nombres de ruta `client.*` → listado/perfil de barberos y reseñas, facturas (listar/descargar), tarjeta de membresía, tienda/carrito/checkout, listado/cancelación de pedidos.
 - `role.custom:barbero` con prefijo `/barbero` y nombres `barber.*` → agenda del día, cambio de estado de citas propias, edición de perfil y horario, portafolio (crear/listar/borrar publicaciones).
- Al final: `require __DIR__.'./auth.php'` — incorpora las rutas estándar de Breeze (login, registro, reseteo de contraseña, verificación de email).

7.2 routes/api.php (todo bajo el prefijo v1)

- El webhook de Stripe vive fuera del prefijo v1 y está explícitamente excluido de la verificación CSRF (Stripe valida su propia firma HMAC en el cuerpo de la petición).
- Ruta de compatibilidad hacia atrás: `availability/slots` (sin el prefijo v1), mantenida por retrocompatibilidad con clientes móviles antiguos.

- Rutas públicas dentro de v1: auth/login (throttle:login), auth/register (throttle:6,1), forgot-password/reset-password, services, barbers, availability/slots, social/feed, chatbot/query (throttle:10,1).
- middleware('mobile.auth') — todo lo que requiere sesión de app móvil: auth/me, logout, refresh-token; perfil (incluye push-token); dashboard; citas (CRUD + updateStatus); pagos (incluye stripe-intent y recibo); clientes (CRUD); servicios (manage CRUD); barberos (manage); usuarios (CRUD); inventario (productos y movimientos); reportes (+ export); configuración (+ maintenance); logs; notificaciones; interacciones sociales; gestión del chatbot (train-history requiere además role.custom:administrador); y los endpoints propios del barbero (barber/me, barber/bio, barber/portfolio, barber/schedule).
- Sub-grupo prefix('admin') + middleware('role.custom:administrador'): dashboard/stats, appointments, revenue, alerts, metrics; gestión completa de barberos (CRUD + horario + clientes regulares + desempeño); clientes (CRUD + segmentación + export); inventario (CRUD + movimiento + resumen + stock bajo); reportes (ingresos/citas/inventario/clientes/personalizado/listado/export); predicciones (ingreso/citas/servicios populares/horas pico/insights).

Detalle técnico de orden de rutas: barbers/{barber} y barbers/{barber}/review se declaran DESPUÉS de barbers/manage precisamente para que el segmento "manage" no sea capturado por el wildcard {barber} — un patrón habitual de Laravel al mezclar rutas estáticas y dinámicas bajo el mismo prefijo, donde el orden de declaración determina cuál coincide primero.

 ESPACIO PARA CAPTURA DE PANTALLA

Salida de "php artisan route:list" filtrada por un módulo (por ejemplo, citas o pagos).

8. Vistas y frontend (Blade + Alpine.js)

122 archivos .blade.php organizados por dominio en resources/views/: appointments, auth, barber, barbers, campaigns, client, clients, components, dashboard.blade.php (en la raíz), emails, errors, inventory, layouts, logs, notifications, payments, pdf (facturas, tarjeta de membresía, recibos), profile, reception, reports, scribe (documentación de API autogenerada), services, settings, social, users, vendor, welcome.blade.php.

8.1 Sistema de navegación

App\Helpers\NavigationMenu es la fuente única de verdad para tres superficies de navegación distintas que de otro modo se desincronizarían fácilmente: el sidebar de escritorio/tablet, el drawer de navegación móvil, y el bottom-nav móvil. Define un diccionario de íconos SVG inline (constante ICONS) indexado por clave semántica — dashboard, wall, appointments, calendar, clients, payments, orders, movements, barbers, users, services, products, settings, reports, campaigns, logs, agenda, schedule, profile, my_appointments, cart, barbers_client, invoices, notifications — y expone un único método sections(?User \$user) que arma dinámicamente las secciones visibles según el rol del usuario autenticado (evaluado con isAdmin()/isReception()/isBarber()/isClient(), todos basados en hasRoleName(), no hasRole()), incluyendo el contador en vivo de artículos en el carrito del cliente (vía CartService::count()).

8.2 Componentes reutilizables (resources/views/components/)

Componente	Función
application-logo	Logo de la marca, reutilizado en header/login/correos
auth-session-status	Mensaje de estado de sesión (ej. "contraseña actualizada")
chatbot	Widget flotante del asistente de IA, embebido en todas las pantallas autenticadas
command-palette	Paleta de comandos estilo Cmd+K para navegación rápida por teclado
danger-button / primary-button / secondary-button	Botones estilizados consistentes
dark-mode-toggle	Interruptor de modo oscuro, persistente entre sesiones
dropdown / dropdown-link	Menús desplegables reutilizables
input-error / input-label / text-input	Elementos de formulario estandarizados
keyboard-shortcuts-help	Panel de ayuda de atajos de teclado
membership-card	Tarjeta de socio con animación de volteo (frente/reverso QR) y confeti al subir de nivel
modal	Ventana modal genérica reutilizable
nav-link	Enlace de navegación con estado activo

Componente	Función
notification-toaster	Notificaciones emergentes (toast) in-app en tiempo real
service-desc	Descripción formateada de un servicio de catálogo
smart-image	Imagen con fallback y carga diferida (lazy load)
toast	Mensaje de confirmación/error temporal

8.3 Patrón visual y accesibilidad

Blade renderizado en servidor + Alpine.js para toda la interactividad (dropdowns, modales, paleta de comandos, flip de tarjeta, toasts) — un patrón deliberadamente ligero frente a una SPA completa, adecuado para un sistema administrativo donde la velocidad de carga inicial y la simplicidad de mantenimiento pesan más que la interactividad tipo aplicación nativa. TailwindCSS utilitario con el plugin `@tailwindcss/forms` para estilos de formulario consistentes. Soporta modo oscuro completo (dark-mode-toggle, con preferencia persistida).

Existe una vista de correo compartida con marca (emails.message, ver sección 6) usada por las 8 notificaciones del sistema, asegurando que cualquier correo que reciba un usuario — sin importar el evento que lo origine — tenga el mismo aspecto visual (logo, colores, tipografía, pie de página).



ESPACIO PARA CAPTURA DE PANTALLA

Sidebar de escritorio en cada uno de los 4 roles (admin, recepción, barbero, cliente).



ESPACIO PARA CAPTURA DE PANTALLA

Paleta de comandos (Cmd+K) abierta, y el modo oscuro activado en cualquier pantalla.

 **ESPACIO PARA CAPTURA DE PANTALLA**

Vista móvil: drawer de navegación y bottom-nav.

9. Seeders y estructura de la base de datos

17 seeders orquestados por `DatabaseSeeder::run()` en un orden estricto de dependencias — cada uno debe correr después del que provee los datos que referencia (por ejemplo, no se pueden crear citas antes de que existan clientes, barberos y servicios):

1. RolePermissionSeeder

Crea 11 permisos reales (`citas.gestionar`, `pagos.gestionar`, `inventario.ver/gestionar`, `clientes.gestionar`, `reportes.ver`, `servicios.gestionar`, `usuarios.gestionar`, `barberos.gestionar`, `configuracion.gestionar`, `logs.ver`) y 4 roles: administrador (todos los permisos), recepcionista (`citas/pagos/clientes/inventario`), barbero y cliente (sin permisos explícitos, autorizados solo por su rol).

2. BarbershopSettingSeeder

Crea la configuración única de la barbería (horarios, política de cancelación, datos de contacto).

3. ServiceSeeder

Puebla el catálogo de servicios (cortes, barba, combos, tratamientos).

4. ProductSeeder

Puebla el catálogo de productos de inventario inicial.

5. AdminUserSeeder

Crea 1 administrador fijo con correo real conocido; la password se toma de `ADMIN_SEED_PASSWORD` o se genera aleatoriamente y se imprime una sola vez en consola. Usa `updateOrCreate()`, por lo que es idempotente (correrlo de nuevo no duplica al administrador).

6. ReceptionUserSeeder

Crea la(s) cuenta(s) de recepcionista, con el mismo patrón seguro de password.

7. BarberSeeder

Genera 50 barberos con datos realistas en español (Faker es_ES), usando inserción masiva (`Model::insert()`) en vez de crear y asignar rol uno por uno — a esa escala, `assignRole()` fila por fila sería demasiado lento. El campo especialidades se guarda deliberadamente como string separado por comas (no un array), porque el resto del código en vistas y controladores lo consume así (`explode(',', ...)`).

8. BarberScheduleSeeder

Genera el horario semanal de cada barbero.

9. ClientSeeder

Genera 1500 clientes en lotes (chunks) de 300, con el mismo patrón de inserción masiva, generando manualmente un `MongoDB\BSON\ObjectId` antes del insert para poder enlazar cada Client con su User correspondiente sin esperar el auto-id de Eloquent.

10. AppointmentSeeder

Genera hasta 150 citas por cliente, distribuidas entre el 1 de enero de 2024 y el fin de la semana actual (evitando domingos). Las citas cuya fecha es futura quedan siempre en estado 'pendiente' (nunca una cita futura aparece ya aprobada); las citas pasadas se distribuyen con pesos realistas: 72% completada, 16% cancelada, 12% no_asistio.

11. PaymentSeeder

Genera un pago por cada cita marcada como completada, con monto/propina/método coherentes.

12. **LoyaltyTransactionSeeder**

Genera el historial de puntos ganados correspondiente a cada cita completada, y recalcula el nivel/puntos/total_citas final de cada cliente.

13. **OrderSeeder**

Genera pedidos de tienda (compras sueltas) y add-ons de producto dentro de citas, con una mezcla realista de estados (entregado/cancelado/pendiente).

14. **WorkSeeder**

Genera publicaciones de portafolio social para cada barbero (2 a 6 por barbero).

15. **WorkImageSeeder**

Genera 1 a 3 imágenes/videos por publicación.

16. **CommentSeeder**

Genera comentarios de un grupo de clientes sobre publicaciones del muro.

17. **ReactionSeeder**

Genera reacciones ("me gusta") sobre las publicaciones del muro.

9.1 Patrones específicos de MongoDB en los seeders

- `role_id` embebido como array de strings en el propio documento del usuario, en vez de una tabla pivote — porque el `MorphToMany` de Spatie Permission no funciona de forma confiable en Mongo (ver sección 3.1).
- Generación manual de `MongoDB\BSON\ObjectId` antes del insert, para poder enlazar relaciones (por ejemplo `Client.user_id`) sin esperar el auto-id que Eloquent asignaría normalmente tras un `create()` individual.
- `Model::insert()` (inserción masiva) en vez de `create()` fila por fila para los seeders de mayor volumen (miles de documentos), con chunking explícito (lotes de 300 o 3000 filas) para no saturar la memoria disponible del proceso PHP.
- `updateOrCreate()` para los seeders de cuentas fijas (administrador, recepción) — hace que esos seeders sean idempotentes: se pueden re-ejecutar sin crear duplicados.

 ESPACIO PARA CAPTURA DE PANTALLA

Salida de "php artisan db:seed" corriendo los 17 seeders en orden.

 **ESPACIO PARA CAPTURA DE PANTALLA**

Vista de las colecciones en MongoDB Compass con sus conteos de documentos.

10. Máquina de estados y reglas de negocio centrales

10.1 Ciclo de vida de una cita (AppointmentStatusService)

Única fuente de verdad de las transiciones válidas de una cita — ningún controlador o vista debe cambiar el campo estado directamente sin pasar por este servicio:

```
pendiente -> confirmada | cancelada confirmada -> en_proceso | completada | no_asistio |
cancelada en_proceso -> completada | cancelada completada, cancelada, no_asistio -> []
(estados terminales)
```

Regla de cobro: CHARGEABLE = ['confirmada', 'en_proceso', 'completada'] — nunca se puede cobrar una cita en estado 'pendiente' sin aprobar. Esta validación se aplica tanto en la web (Payment\PaymentController) como en la API (Api\PaymentController).

Mapa de roles por transición: confirmada, en_proceso, completada y no_asistio solo las puede fijar el barbero (únicamente sobre sus propias citas, validado en el controlador), la recepcionista o el administrador. cancelada además puede fijarla el propio cliente dueño de la cita.

Métodos expuestos por el servicio: canTransition(\$from,\$to) (booleano), allowedFor(\$from) (lista de estados destino válidos), roleCanSet(\$to,\$role) (booleano), isChargeable(\$estado) (booleano), transition(\$appointment,\$to) — valida la legalidad de la transición y lanza InvalidAppointmentTransitionException si no es válida; al cancelar, además marca el timestamp cancelada_en. El servicio deliberadamente NO dispara efectos secundarios (notificaciones, otorgamiento de puntos de lealtad) — esa responsabilidad recae en quien invoca la transición (el controlador), manteniendo el servicio enfocado en una sola responsabilidad.

10.2 Notificación en cascada (AppointmentNotifier)

Reparte notificaciones diferenciadas por rol en 3 momentos del ciclo de vida de una cita: created() (al crearse — notifica al barbero asignado y, según preferencias, al cliente), cancelled() (incluye información sobre el origen de la cancelación — quién la canceló y por qué), y statusChanged() (con un mapa de mensajes específico por estado destino: completada, confirmada, en_proceso, no_asistio, cada uno con su propio texto y tono). Inmediatamente después de marcar una cita como 'completada', se agenda ReviewRequestNotification con un retraso (delay) de 2 horas — el tiempo suficiente para que el cliente ya haya salido del establecimiento antes de pedirle una reseña.

10.3 Niveles de lealtad (LoyaltyService)

Nivel	Etiqueta de marketing	Umbral (citas completadas)	Descuento
nuevo	Caballero	0	0%
regular	Regular	5	5%
vip	V.I.P	10	10%
leyenda	Leyenda	20	15%

10 puntos se otorgan por cada cita completada, 5 puntos por cada reseña dejada.

redeemPoints(\$cantidad) valida que el saldo actual sea suficiente ANTES de descontar — nunca se

permite un saldo negativo de puntos — y registra el movimiento como una `LoyaltyTransaction` de tipo 'canjeado' (auditable). Al subir de nivel se dispara automáticamente `LoyaltyNotification` y, del lado del frontend, se dispara la animación de confeti junto con el volteo automático de la tarjeta de membresía para mostrar el nuevo nivel.

10.4 Flujo de pedidos (tienda vs. add-on de cita)

`CartService` acumula líneas de producto en la sesión HTTP, respetando en todo momento el stock máximo disponible de cada producto (no permite agregar más unidades de las que hay en existencia, evitando sobreventa desde el propio carrito). Al confirmar la compra, `OrderService::place()` primero valida el stock de TODOS los ítems del pedido antes de descontar ninguno — esto evita el escenario de quedarse a medio proceso (descontar 2 de 3 productos y fallar en el tercero, dejando el inventario inconsistente). Registra un movimiento de inventario tipo 'salida' por cada línea de forma transaccional (con bloqueo de fila `lockForUpdate` para evitar condiciones de carrera si dos clientes compran el mismo producto casi simultáneamente), genera un folio único legible `P-XXXXXX`, y crea el `Order` con `tipo='tienda'` (compra suelta) o `tipo='cita'` (si se agregó como add-on dentro de una reserva, en cuyo caso incluye el `appointment_id` de origen). `cancel()` solo puede actuar sobre pedidos en estado 'pendiente' y revierte el stock descontado con movimientos de tipo 'entrada'.

10.5 Despacho de campañas de marketing

`CampaignDispatcher::audienceUserIds()` resuelve el segmento declarado de una campaña ('todos' o un nivel de lealtad específico) a la lista concreta de IDs de usuario a los que aplica, consultando la colección `clients` filtrada por nivel. `dispatch()` construye una `PromotionNotification` (que internamente exige que el destinatario tenga activado el opt-in 'promociones' en sus preferencias, y que añada tracking de apertura/clic automáticamente si la notificación proviene de una `Campaign`) y la envía en lotes de 200 destinatarios a la vez usando `Notification::send()`, evitando encolar miles de trabajos de una sola vez. Al terminar, marca la campaña como 'enviada' registrando el conteo real de destinatarios y la fecha/hora de envío. El comando programado `campaigns:dispatch-due` (corre cada 5 minutos vía el scheduler) reutiliza exactamente el mismo `CampaignDispatcher` para procesar las campañas programadas cuya fecha programada para ya se cumplió, sin necesitar lógica de envío duplicada.

10.6 Tarjeta de membresía digital

`MemberCardService::memberNumber()` deriva un número de socio legible y estable a partir del `ObjectId` único e inmutable del usuario (formato `"UB · XXXX XXXX"`), de modo que el mismo usuario siempre obtiene el mismo número visible sin necesidad de almacenarlo por separado. `memberSince()` toma directamente el año del campo `created_at` de la cuenta. `qrDataUri()/qrPngDataUri()` generan un código QR (en formato SVG o PNG, codificado como data-URI listo para insertarse directamente en HTML o en un PDF sin archivos intermedios) cuyo contenido es el payload `UB|{id}|{name}` — un formato de verificación simple. Este servicio alimenta tanto la vista web de la tarjeta (con animación de volteo hacia el QR y confeti al subir de nivel) como el PDF descargable generado por `Client\MembershipController::card`.

10.7 No-show automático y recordatorios

El comando `appointments:mark-no-show` (corre cada hora) revisa las citas en estado 'confirmada' cuyo horario ya pasó sin que nadie las haya marcado manualmente como completada o cancelada, y las

transiciona automáticamente a 'no_asistio' a través del mismo AppointmentStatusService — garantizando que ninguna cita quede indefinidamente en un estado "limbo" que ya no refleja la realidad. El comando appointments:send-reminders (corre cada 10 minutos) envía recordatorios escalonados (24 horas y 2 horas antes de la cita), usando los timestamps reminder_24h_sent_at/reminder_2h_sent_at del propio modelo Appointment para garantizar que cada recordatorio se envíe como máximo una sola vez, incluso si el comando se ejecuta con más frecuencia de la necesaria.



ESPACIO PARA CAPTURA DE PANTALLA

Diagrama de la máquina de estados de citas (pendiente → confirmada → ... → completada).



ESPACIO PARA CAPTURA DE PANTALLA

Tarjeta de membresía en la app, mostrando el frente (nivel/puntos) y el reverso (QR).

11. Seguridad

11.1 Autenticación y autorización

- Web: sesión de Laravel (Breeze) sobre el driver database — las sesiones se persisten en una colección de Mongo, no solo en cookies firmadas, permitiendo invalidar sesiones desde el servidor si fuera necesario.
- API móvil: token Bearer propio (MobileApiToken), emitido por `User::issueMobileApiToken()`. El token en claro se entrega una sola vez al cliente; el servidor solo guarda su hash SHA-256 — si la base de datos se filtrara, los tokens no podrían reconstruirse ni reutilizarse directamente.
- Autorización de dos capas: `role.custom:<roles>` y `permission.custom:<permisos>`, ambos implementados con `User::hasRoleName()/roleNames()` para evitar la fragilidad de `hasRole()/hasAnyRole()` de Spatie sobre MongoDB (ver sección 3.1).
- El middleware `CheckMaintenanceMode` permite el acceso de administradores incluso con el modo mantenimiento activado, para que puedan desactivarlo sin quedar bloqueados fuera de su propio sistema.

11.2 Protección de rutas y CSRF

- Todas las rutas web mutables (POST/PUT/PATCH/DELETE) están protegidas por el token CSRF estándar de Laravel, excepto el webhook de Stripe, que en su lugar valida la firma HMAC que Stripe adjunta a cada petición del webhook.
- Las rutas de tracking de campañas (`/t/o`, `/t/c`) son deliberadamente públicas (un correo no puede incluir un token CSRF de sesión), pero solo exponen operaciones de lectura/registro de evento, nunca de escritura de datos sensibles.

11.3 Datos sensibles

- Las passwords de las cuentas sembradas (admin/recepción/barberos/clientes) nunca se hardcodean en el código fuente — se leen de variables de entorno opcionales o se generan aleatoriamente y se imprimen una sola vez, evitando que terminen en el historial de git de un repositorio público.
- Los tokens de API móvil se almacenan hasheados (SHA-256), nunca en texto plano.
- Los datos bancarios de la configuración del negocio (`BarbershopSetting.datos_bancarios`) se manejan como un array, accesible únicamente con `permission.custom:configuracion.gestionar`.

11.4 Rate limiting

- Login: `throttle:login` (limita intentos de fuerza bruta).
- Registro: `throttle:6,1` (máximo 6 intentos por minuto).
- Chatbot público: `throttle:10,1` en la API, más un rate limiting propio configurado en `config/chatbot.php` (`chatbot.rate_limit.max_attempts/decay_seconds`) para controlar el costo de las llamadas al proveedor de IA.

11.5 Monitoreo

Sentry (sentry-laravel) captura excepciones no controladas en producción, dando visibilidad sobre errores reales de usuarios sin depender únicamente de que alguien reporte el problema manualmente. La bitácora de auditoría (Spatie Activitylog) registra automáticamente los cambios sobre los modelos críticos (Appointment, InventoryMovement, Payment, Product, User), visibles en la pantalla de Bitácora para el administrador.

A large rectangular area with a dashed border, intended for a screenshot. Inside, there is a small icon of a folder with a document and the text "ESPACIO PARA CAPTURA DE PANTALLA".

 **ESPACIO PARA CAPTURA DE PANTALLA**

Pantalla de Bitácora de auditoría (Log\ActivityLogController) mostrando eventos registrados.

12. Manejo de errores, logging y rendimiento

12.1 Excepciones de dominio

El proyecto define excepciones específicas de negocio en vez de usar excepciones genéricas, lo que permite capturarlas selectivamente y mostrar mensajes claros al usuario final:

`ClientAlreadyBookedException` (un cliente ya tiene otra cita el mismo día),

`AppointmentConflictException` (un barbero tiene solape de horario),

`InvalidAppointmentTransitionException` (se intentó una transición de estado no permitida),

`InsufficientStockException` (no hay stock suficiente para una salida de inventario).

12.2 Notificaciones resilientes

`AppointmentNotifier` envuelve cada envío de notificación en un bloque `try/catch`, registrando cualquier fallo con `Log::warning()` en vez de dejar que la excepción se propague — un fallo al enviar un correo de confirmación nunca debe impedir que la cita se guarde correctamente. Este patrón se repite en `MessagingService` (Twilio): si no hay credenciales configuradas, simula el envío y lo registra en el log en vez de lanzar una excepción.

12.3 Colas y procesamiento asíncrono

Todas las notificaciones implementan `ShouldQueue`, procesadas por el contenedor worker (`php artisan queue:work --tries=3 --timeout=90` — reintenta hasta 3 veces si un trabajo falla, con un límite de 90 segundos por intento antes de considerarlo colgado). Esto desacopla la generación de un evento (crear una cita, completar un pago) de las tareas potencialmente lentas que dispara (enviar un correo con PDF adjunto, contactar la API de Twilio), manteniendo las respuestas HTTP rápidas para el usuario.

12.4 Concurrencia en inventario

`InventoryService::registerMovement()` y `OrderService::place()` usan transacciones de base de datos con `lockForUpdate()` sobre el documento del producto — esto evita condiciones de carrera cuando dos operaciones intentan modificar el mismo stock casi al mismo tiempo (por ejemplo, dos ventas simultáneas del mismo producto con solo 1 unidad restante): sin este bloqueo, ambas operaciones podrían leer "1 disponible" y ambas completar la venta, dejando el stock en -1.

12.5 Caché y colas sobre Redis

`CACHE_STORE=redis` y `QUEUE_CONNECTION=redis` comparten el mismo motor Redis para dos propósitos distintos (caché de aplicación y cola de trabajos), simplificando la infraestructura de despliegue al no requerir dos sistemas separados.

12.6 Estado actual de pruebas automatizadas

Nota honesta: el proyecto no cuenta actualmente con un directorio `tests/` ni una suite de pruebas automatizadas activa. La verificación de cambios se realiza manualmente (navegador + revisión de logs de Docker). Se recomienda como trabajo futuro reconstruir una suite de pruebas de integración,

priorizando la máquina de estados de citas (sección 10.1), el flujo de pedidos/inventario (sección 10.4, por su sensibilidad a condiciones de carrera) y el sistema de roles/permisos (sección 3.1, por su historial de fragilidad sobre MongoDB).



ESPACIO PARA CAPTURA DE PANTALLA

Panel de Sentry mostrando la captura de una excepción, o los logs del worker procesando la cola.

13. Configuración, comandos artisan y tareas programadas

13.1 Archivos de configuración relevantes (config/)

app.php, auth.php, cache.php, database.php (conexión mongodb), session.php (driver database sobre Mongo), queue.php (redis), mail.php, logging.php, permission.php (Spatie Permission), activitylog.php (Spatie Activitylog), dompdf.php, excel.php, scribe.php (documentación de API), sentry.php, services.php, filesystems.php, y chatbot.php — configuración propia del asistente de IA: proveedor en cascada (ai.provider), parámetros de conexión a Ollama, rate limiting propio del chatbot, y telemetría con costo estimado por cada 1,000 tokens procesados (útil para monitorear el gasto si se usa el proveedor en la nube).

13.2 Comandos artisan personalizados (app/Console/Commands/)

Comando	Propósito
tokens:clean-expired	Limpia tokens de API móvil expirados de la colección mobile_api_tokens
campaigns:dispatch-due	Despacha campañas de marketing programadas cuya fecha ya se cumplió
loyalty:draw-raffle	Ejecuta el sorteo mensual de lealtad entre los clientes elegibles
appointments:mark-no-show	Marca automáticamente como 'no_asistio' las citas confirmadas cuyo horario ya pasó
appointments:send-reminders	Envía recordatorios escalonados de citas próximas (24h y 2h antes)
reports:daily-summary	Genera y envía el resumen diario del negocio al administrador
inventory:low-stock-alert	Genera la alerta de stock bajo para el administrador
SendPromotionCommand (campaigns:promote)	Envío manual e inmediato de una promoción fuera del flujo de campañas programadas
TestLearningSystem	Utilidad interna de pruebas del sistema de aprendizaje del chatbot
ValidateUserRoles	Comando de auditoría y reparación: detecta usuarios sin rol asignado (el mismo tipo de incidente documentado en la Unidad VI del módulo de analítica) y permite corregirlo

13.3 Tareas programadas (routes/console.php)

Laravel 12 ya no usa el clásico app/Console/Kernel.php para definir el scheduler — las tareas se declaran directamente en routes/console.php:

```
appointments:send-reminders -> everyTenMinutes() tokens:clean-expired -> daily()
loyalty:draw-raffle -> monthlyOn(1, '08:00') inventory:low-stock-alert ->
dailyAt('09:00') reports:daily-summary -> dailyAt('21:30') campaigns:dispatch-due
-> everyFiveMinutes() appointments:mark-no-show -> hourly()
```

El contenedor scheduler ejecuta php artisan schedule:work de forma continua (en vez de depender de un cron del sistema operativo host) para disparar estas tareas en el momento correcto; el contenedor

worker procesa en paralelo las colas de Redis que estas tareas generan (envío de correos, notificaciones, generación de PDFs).

 ESPACIO PARA CAPTURA DE PANTALLA

Logs del contenedor scheduler o worker ejecutando una tarea programada en tiempo real.

14. Guía de resolución de problemas comunes

Síntoma	Causa probable y solución
Un usuario no puede acceder a su panel aunque tiene el rol correcto	hasRole()/hasAnyRole() de Spatie es poco confiable sobre Mongo (sección 3.1). Verificar con User::hasRoleName() en tinkers; si el controlador o middleware en cuestión todavía usa hasRole(), migrarlo a hasRoleName()/roleNames().
Los correos no se envían realmente (solo aparecen en el log)	MAIL_MAILER=log es el valor por defecto en desarrollo. Cambiar a smtp y usar MAIL_SCHEME=smtp (587) o smtps (465) — nunca tls a secas, que Laravel 12 rechaza.
El chatbot no responde o tarda mucho	Verificar que Ollama esté corriendo en el host y accesible en OLLAMA_URL=http://host.docker.internal:11434; revisar OLLAMA_TIMEOUT si el modelo es lento en el hardware disponible.
Un producto queda con stock negativo	Señal de que una operación de inventario no pasó por InventoryService::registerMovement() (que usa lockForUpdate) — revisar que ningún código nuevo modifique stock_actual directamente.
Una cita queda "pendiente" indefinidamente en el pasado	El comando appointments:mark-no-show corre cada hora vía el scheduler; verificar que el contenedor scheduler esté corriendo (docker compose ps) y que su healthcheck esté en estado healthy.
Las notificaciones no llegan aunque el correo esté bien configurado	Revisar las preferencias de notificación del usuario (User::wantsNotificationChannel()) — puede tener el canal desactivado explícitamente; revisar también que el contenedor worker esté procesando la cola (los envíos son asíncronos).
Error de "MorphToMany is not supported" o similar en logs relacionados con roles	Síntoma directo de intentar usar la relación roles() de Spatie de forma incompatible con Mongo (queries con joins/wheres complejos) — usar User::scopeWhereRoleName() o roleNames() en su lugar.
El healthcheck del worker o scheduler aparece unhealthy	Estos contenedores NO escuchan el puerto 9000 (heredado del Dockerfile de app) — su healthcheck real usa pgrep sobre el proceso artisan correspondiente; verificar que el proceso realmente esté corriendo dentro del contenedor con docker exec ... ps aux.

 ESPACIO PARA CAPTURA DE PANTALLA

Terminal mostrando el diagnóstico de alguno de estos problemas (por ejemplo, tinkers verificando un rol).

15. Glosario de términos técnicos del proyecto

Término	Significado en el contexto de UrbanBlade
role_id	Campo embebido en el documento de User que guarda directamente los IDs de rol asignados, evitando depender de la relación MorphToMany de Spatie sobre MongoDB
role.custom / permission.custom	Middlewares propios de autorización por rol/permiso, implementados para ser confiables sobre MongoDB
mobile.auth	Middleware de autenticación de la API móvil basado en MobileApiToken, no en Laravel Sanctum
Add-on de cita	Producto de tienda agregado dentro del mismo flujo de reserva de una cita (Order con tipo='cita')
Walk-in	Cliente que llega sin cita previa; se registra una cita rápida desde recepción (AppointmentController::walkIn)
Estado terminal	Estado de una cita que ya no permite ninguna transición posterior: completada, cancelada, no_asistio
CHARGEABLE	Constante de AppointmentStatusService que define en qué estados una cita puede cobrarse
Segmento (campañas)	Criterio de audiencia de una campaña de marketing: 'todos' o un nivel de lealtad específico
Folio	Código único legible de un pedido (formato P-XXXXXX) o de un pago (formato F-XXXXXX)
Bulk insert	Inserción masiva de documentos (Model::insert()) usada en los seeders de gran volumen, en vez de crear cada registro individualmente
Bitácora / Activity log	Registro de auditoría automático (Spatie Activitylog) sobre los modelos críticos del sistema
Slot disponible	Bloque de horario libre calculado por AppointmentService::getAvailableSlots() para agendar una cita nueva

16. Anexos

Anexo A — Checklist de despliegue a producción

1. Confirmar APP_ENV=production y APP_DEBUG=false en el .env real.
2. Verificar que MONGODB_URI apunte al clúster de producción de MongoDB Atlas, no al de desarrollo.
3. Configurar MAIL_MAILER real (no log) con MAIL_SCHEME=smtp o smtps.
4. Configurar las credenciales reales de Twilio si se requiere SMS/WhatsApp en producción.
5. Configurar SENTRY_LARAVEL_DSN para monitoreo de errores en vivo.
6. Ejecutar php artisan migrate --force y, si es la primera vez, php artisan db:seed con las passwords de seeders fijadas explícitamente por variable de entorno.
7. Verificar que los 6 servicios de Docker Compose reporten estado healthy (docker compose ps).
8. Verificar que el scheduler y el worker estén realmente corriendo (no solo el contenedor "up", sino el proceso artisan dentro de él).
9. Confirmar que el modo mantenimiento (BarbershopSetting.maintenance_mode) esté desactivado antes de anunciar el lanzamiento.

Anexo B — Resumen de puertos expuestos

Puerto	Servicio
8000	Aplicación web (Nginx)
8025	Interfaz de Mailpit (solo desarrollo)
1025	SMTP de Mailpit (solo desarrollo)
9000	PHP-FPM (interno, no expuesto al host salvo por Nginx)

Anexo C — Resumen de roles y permisos

Rol	Permisos asignados
administrador	citas.gestionar, pagos.gestionar, inventario.ver, inventario.gestionar, clientes.gestionar, reportes.ver, servicios.gestionar, usuarios.gestionar, barberos.gestionar, configuracion.gestionar, logs.ver (todos)
repcionista	citas.gestionar, pagos.gestionar, clientes.gestionar, inventario.ver, inventario.gestionar
barbero	ninguno explícito — autorizado por role.custom:barbero sobre sus propios recursos
cliente	ninguno explícito — autorizado por role.custom:cliente sobre sus propios recursos

 ESPACIO PARA CAPTURA DE PANTALLA

Pantalla de gestión de roles/permisos en el panel de administración (si existe una UI para editarlos).

17. Ejemplos de peticiones a la API (v1)

Esta sección ilustra el formato real de petición/respuesta para los endpoints más usados por el cliente móvil, como referencia rápida para quien integre contra la API sin tener que leer el controlador completo.

17.1 Autenticación

```
POST /api/v1/auth/login Body: { "email": "cliente@ejemplo.com", "password": "....." }
Respuesta 200: { "token": "1a2b3c...", // usar como Authorization: Bearer <token>
"user": { "id": "...", "name": "...", "email": "...", "role": "cliente" } }
```

17.2 Consultar disponibilidad

```
GET /api/v1/availability/slots?barber_id=...&service_id=...&fecha=2026-07-20 Respuesta 200: {
"slots": ["09:00", "09:30", "10:00", "11:30", "16:00"] }
```

17.3 Crear una cita (autenticado)

```
POST /api/v1/appointments Headers: Authorization: Bearer <token> Body: { "barber_id": "...",
"service_id": "...", "fecha": "2026-07-20", "hora_inicio": "10:00" } Respuesta 201: {
"appointment": { "id": "...", "estado": "pendiente", ... } } Respuesta 422 (conflicto): {
"message": "El cliente ya tiene una cita ese día." }
```

17.4 Cambiar el estado de una cita

```
PATCH /api/v1/appointments/{id}/status Body: { "estado": "confirmada" } Respuesta 200: {
"appointment": { "id": "...", "estado": "confirmada" } } Respuesta 403: { "message": "Esta
transición no está permitida para tu rol." }
```

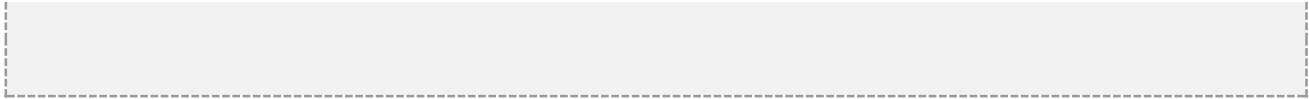
17.5 Registrar un pago

```
POST /api/v1/payments Body: { "appointment_id": "...", "monto": 350.00, "metodo_pago":
"efectivo", "propina": 30.00 } Respuesta 201: { "payment": { "id": "...", "comprobante_pdf":
"/storage/..." } } Respuesta 422: { "message": "Esta cita no está en un estado cobrable." }
```

17.6 Feed social

```
GET /api/v1/social/feed Respuesta 200: { "works": [ { "id": "...", "barbero": "...",
"title": "...", "images": [...], "reactions_count": 12, "comments_count": 3 } ] }
```

 ESPACIO PARA CAPTURA DE PANTALLA



Cliente HTTP (Postman/Insomnia) mostrando una petición real a la API con su respuesta.

18. Convenciones de código y guía de extensión

18.1 Convenciones generales

- Los controladores nunca contienen lógica de negocio compleja — delegan a un servicio de `app/Services/`. Un controlador debe poder leerse y entenderse en menos de un minuto.
- Las reglas de autorización se expresan siempre con los middlewares `role.custom/permission.custom`, nunca con condicionales `if($user->hasRole(...))` dispersos en las vistas o controladores — esto centraliza el control de acceso y facilita auditarlo.
- Toda excepción de negocio (ver sección 12.1) se declara como una clase propia, nunca como un `throw new Exception('...')` genérico — permite capturarla selectivamente y dar un mensaje claro al usuario.
- Los seeders de volumen usan inserción masiva (`Model::insert()`) con chunking; los seeders de cuentas fijas usan `updateOrCreate()` para ser idempotentes.
- Las notificaciones siempre implementan `ShouldQueue` y respetan las preferencias de canal del usuario — ninguna notificación nueva debe ignorar `wantsNotificationChannel()`.

18.2 Cómo agregar una nueva notificación

1. Crear la clase en `app/Notifications/` implementando `ShouldQueue`, con métodos `via($notifiable)` (consultando `wantsNotificationChannel()`), `toMail()` (usando la vista compartida `emails.message`) y, si aplica, `toDatabase()`.
2. Si debe enviarse por SMS/WhatsApp, implementar además `toTwilio()` y agregar el canal `Channels\TwilioChannel` a la lista de `via()`.
3. Disparar la notificación desde el servicio de dominio correspondiente (nunca desde el controlador directamente), usando `Notification::send()` o `$modelo->notify()`.
4. Si la notificación debe respetar un opt-in específico (como 'promociones'), validarlo explícitamente dentro de `via()` antes de incluir el canal mail.

18.3 Cómo agregar un nuevo tipo de reporte

1. Agregar el método correspondiente en `ReportService` (por ejemplo, `nuevoReporte()`).
2. Registrar el nuevo tipo en el dispatcher `reportByType()` del mismo servicio.
3. Exponerlo en `Report\ReportController` (web) y, si aplica, en `Api\ReportController` / `Api\Admin\ReportAdminController`.
4. Agregar la opción correspondiente en la vista de selección de reportes (`resources/views/reports/`).
5. Si el reporte debe graficarse, devolver también un bloque chart (labels/values) desde el método del paso 1 — `reportByType()` ya lo incluye en el array de salida y las tres vistas (web con `Chart.js`, Excel con `WithCharts` de `PhpSpreadsheet`, y PDF con un gráfico en HTML/CSS porque `DomPDF` no ejecuta JavaScript) lo consumen del mismo lugar sin código adicional.

18.4 Cómo agregar un nuevo estado o transición de cita

Precaución: modificar la máquina de estados de citas (sección 10.1) afecta directamente el flujo de cobro, las notificaciones y el programa de lealtad. Cualquier cambio aquí debe revisarse con cuidado antes de desplegarse.

1. Agregar el nuevo estado a la constante TRANSITIONS de AppointmentStatusService, definiendo explícitamente a qué estados puede pasar.
2. Decidir si el nuevo estado debe agregarse a CHARGEABLE (si permite cobrar la cita).
3. Actualizar ROLE_MAP para definir qué roles pueden fijar ese estado.
4. Agregar el mensaje correspondiente en el mapa de textos de AppointmentNotifier::statusChanged(), y en las vistas que muestran el estado con una etiqueta visual (badge de color).

